

Layouts responsivos e front-end

Tópico: Visão geral da aula

Nesta aula, estudaremos como páginas web se adaptam a diferentes telas e como JavaScript adiciona comportamento básico ao front-end. A ideia é ligar HTML, CSS e programação: estruturar conteúdo, organizar layout, criar navegação clara e responder a ações do usuário.

[Link: MDN Responsive Design](#)



Objetivos de aprendizagem

Tópico: O que o aluno deverá compreender

Ao final da aula, o estudante deverá explicar responsividade, diferenciar layout fixo, fluido e adaptável, construir navegação adequada a computador e celular, aplicar media queries e compreender variáveis, funções, estruturas de controle e eventos em JavaScript para interações simples.

Conceitos-chave

1

responsividade

2

navegação

3

media queries

4

eventos JS

usar em protótipos

Retomando HTML, CSS e JavaScript

Tópico: Separação de responsabilidades

HTML descreve a estrutura e o significado do conteúdo; CSS define apresentação, layout, espaçamentos, cores e tipografia; JavaScript adiciona comportamento. Essa separação facilita manutenção, acessibilidade e evolução do projeto, pois cada tecnologia assume uma responsabilidade principal na página.

Três camadas

1. HTML: estrutura

2. CSS: aparência

3. JS: comportamento

4. Navegador interpreta

5. Usuário interage

[Link: MDN HTML](#)

O que é design responsivo?

Tópico: Adaptação ao contexto de uso

Design responsivo é a capacidade de uma página reorganizar conteúdo conforme largura da tela, orientação, recursos do dispositivo e preferências do usuário. O objetivo é manter leitura, navegação e operação confortáveis em celulares, tablets, notebooks e monitores maiores.



[Link: web.dev Responsive Basics](https://web.dev/Responsive-Basics)

Por que responsividade importa?

Tópico: Uso real em múltiplos dispositivos

Usuários acessam sistemas acadêmicos, formulários, catálogos e serviços por telas muito diferentes. Uma página que funciona apenas no monitor do desenvolvedor pode ficar difícil de ler no celular, esconder botões importantes ou tornar a navegação lenta e confusa.



Layout fixo, fluido e responsivo

Tópico: Três formas de organizar a página

Um layout fixo usa larguras rígidas e pode causar rolagem horizontal. Um layout fluido usa porcentagens e se ajusta melhor ao espaço. Um layout responsivo combina fluidez, limites, media queries e reorganização de blocos conforme a necessidade da interface.

Conceitos-chave

F

fixo

%

fluido

@

media query

R

responsivo

responsivo combina técnicas

Viewport e meta viewport

Tópico: A tela vista pelo navegador

Viewport é a área disponível para exibir a página no navegador. Em páginas móveis, a tag meta viewport orienta o navegador a usar a largura real do dispositivo. Sem ela, a página pode ser renderizada como uma versão reduzida de desktop.

[Link: MDN Viewport](#)

Meta viewport

```
<meta name="viewport"
      content="width=device-width,
              initial-scale=1.0">
```

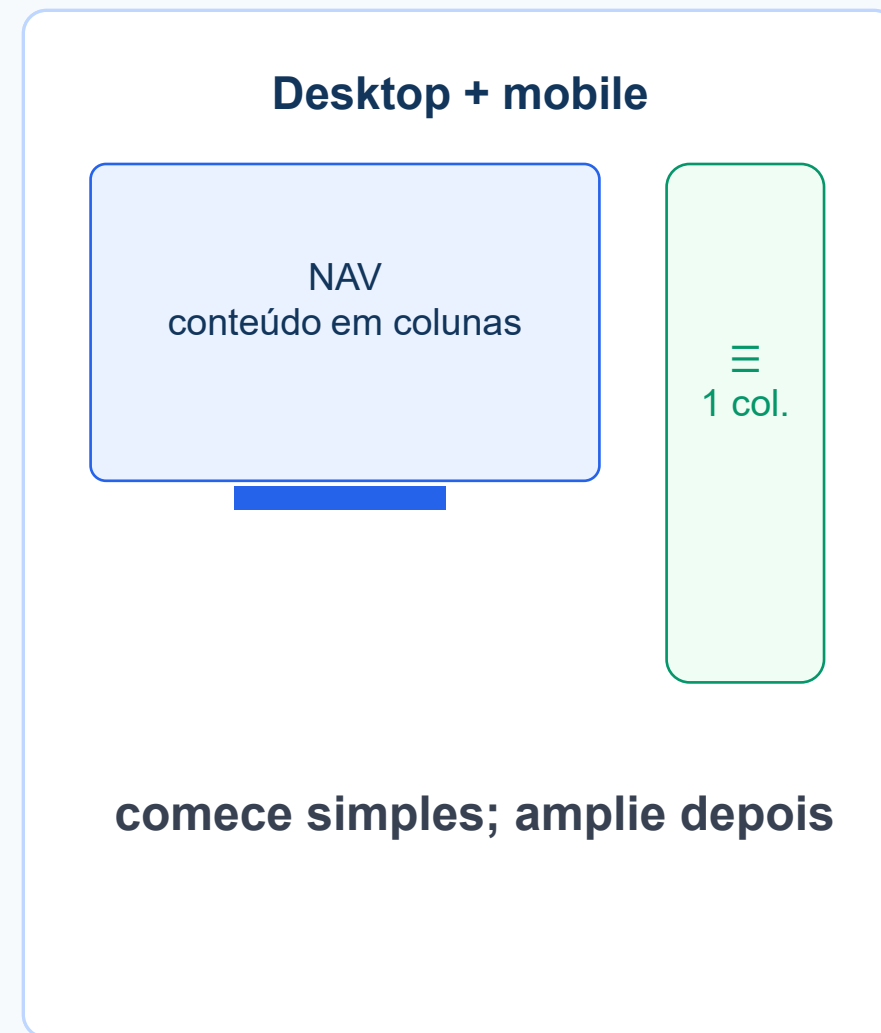
[editar](#) → [salvar](#) → [testar](#)

Mobile first

Tópico: Começar pela menor tela

Mobile first é uma abordagem em que o CSS começa atendendo telas pequenas e depois acrescenta melhorias para telas maiores. Essa estratégia ajuda a priorizar conteúdo essencial, reduz excesso visual e evita que a versão móvel seja apenas uma adaptação improvisada do desktop.

[Link: MDN Responsive Design](#)



Unidades relativas no CSS

Tópico: Tamanhos flexíveis

Unidades como %, rem, em, vw e vh ajudam elementos a acompanhar o contexto da tela. Pixels continuam úteis, mas layouts responsivos dependem de combinações como width: 100%, max-width, gap e font-size relativo para manter leitura confortável.

Unidades úteis

```
.container {  
  width: min(92%, 960px);  
}  
.card {  
  padding: 1rem;  
}  
h1 {  
  font-size: clamp(1.6rem,  
    4vw, 2.8rem);  
}
```

[editar](#) → [salvar](#) → [testar](#)

Imagens e mídias flexíveis

Tópico: Evitar estouro visual

Imagens muito largas podem quebrar o layout em telas pequenas. Uma regra comum é definir `max-width: 100%` e `height: auto` para que a mídia acompanhe o contêiner sem distorção. Também é importante manter texto alternativo quando a imagem informa algo.

[Link: MDN Images in HTML](#)

Imagem responsiva

```
img, video {  
  max-width: 100%;  
  height: auto;  
}  
figure {  
  margin: 0;  
}
```

[editar](#) → [salvar](#) → [testar](#)

Flexbox: alinhamento em uma dimensão

Tópico: Linhas, colunas e espaçamento

Flexbox organiza elementos em linha ou coluna e facilita alinhamento, distribuição de espaço e mudança de direção. É muito útil para menus, grupos de botões, cards simples e cabeçalhos. Em telas pequenas, a direção pode mudar para facilitar leitura.

Flexbox

```
.menu {  
  display: flex;  
  gap: 1rem;  
  flex-wrap: wrap;  
}  
@media (max-width: 600px) {  
  .menu {  
    flex-direction: column;  
  }  
}
```

[editar](#) → [salvar](#) → [testar](#)

[Link: MDN Flexbox](#)

CSS Grid: organização em duas dimensões

Tópico: Linhas e colunas

CSS Grid é indicado quando precisamos controlar linhas e colunas ao mesmo tempo. Ele ajuda a criar áreas de conteúdo, grades de cards e estruturas mais completas. Com repeat, minmax e auto-fit, é possível criar grades que se ajustam automaticamente ao espaço disponível.

Grid responsivo

```
.cards {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit,  
      minmax(220px, 1fr));  
  gap: 1rem;  
}
```

[editar](#) → [salvar](#) → [testar](#)

[Link: MDN CSS Grid](#)

Media queries

Tópico: CSS condicionado ao ambiente

Media queries permitem aplicar regras apenas quando certas condições são atendidas, como largura mínima, orientação ou preferência do usuário. Na prática, elas ajudam a mudar navegação, número de colunas, espaçamentos e tamanhos sem alterar o HTML.

Media query

```
@media (min-width: 768px) {  
  .layout {  
    display: grid;  
    grid-template-columns: 2fr  
1fr;  
  }  
}
```

[editar](#) → [salvar](#) → [testar](#)

[Link: MDN Media Queries](#)

Breakpoints

Tópico: Pontos de mudança do layout

Breakpoint é o ponto em que o layout precisa mudar para continuar funcionando bem. Em vez de escolher valores apenas por modelos de celulares, observe quando o conteúdo começa a ficar apertado, longo, ilegível ou difícil de navegar.

Escolha de breakpoint

1. testar em largura menor

2. identificar quebra

3. ajustar regra CSS

4. testar teclado

5. validar novamente

Navegação responsiva

Tópico: Menu que continua utilizável

A navegação deve permitir que o usuário encontre áreas importantes da página em qualquer tela. No desktop, links horizontais podem funcionar bem. No celular, talvez seja melhor empilhar links, reduzir espaçamentos ou usar um botão de menu claramente identificado.

[Link: MDN Navigation](#)

Navegação responsiva

Cabeçalho

Início

Serviços

Contato

≡
**menu em telas
pequenas
links alcançáveis por
teclado**

Menu hambúrguer com cuidado

Tópico: Interação e acessibilidade

O menu hambúrguer pode economizar espaço, mas precisa ser claro, acessível por teclado e compreensível por leitores de tela. O botão deve indicar o estado do menu, abrir e fechar corretamente, e os links devem permanecer alcançáveis em ordem lógica.

Botão de menu

```
<button id="btn-menu"
  aria-expanded="false"
  aria-controls="menu">
  Menu
</button>
<nav id="menu">...</nav>
```

[editar](#) → [salvar](#) → [testar](#)

[Link: W3C WCAG 2.2](#)

Adaptação a toque, mouse e teclado

Tópico: Entrada em dispositivos diferentes

Usuários podem interagir por mouse, toque, teclado ou tecnologias assistivas. Botões precisam ter área suficiente, foco visível e ordem lógica. Uma interface responsiva deve continuar operável quando o usuário navega por Tab, Enter, Espaço e setas.

Conceitos-chave

Tab

teclado

Tap

toque



mouse

A11y

assistiva

testar sem mouse

[Link: WCAG Quick Reference](#)

DevTools para testar responsividade

Tópico: Ferramenta prática gratuita

Os navegadores modernos possuem ferramentas de desenvolvedor que simulam larguras de tela, inspecionam HTML, alteram CSS temporariamente e ajudam a diagnosticar problemas. No laboratório, DevTools é essencial para testar layout antes de entregar a atividade.

Teste no navegador

1. abrir página

2. inspecionar elemento

3. ativar modo responsivo

4. alterar CSS temporário

5. corrigir no arquivo

[Link: Chrome DevTools](#)

JavaScript no front-end

Tópico: Comportamento da página

JavaScript permite reagir a ações do usuário, alterar conteúdo, validar dados de forma complementar e controlar pequenos estados da interface. Ele roda no navegador e trabalha junto com o DOM, que representa a página HTML como uma estrutura manipulável.

JS na página

1. HTML cria DOM

2. CSS apresenta

3. JS escuta eventos

4. JS altera estado

5. Usuário percebe resposta

[Link: MDN JavaScript](#)

Variáveis

Tópico: Guardar valores para usar depois

Variável é um nome associado a um valor. No front-end, variáveis podem guardar texto digitado, número de itens, estado do menu ou mensagem de erro. Em JavaScript moderno, usamos `let` para valores que mudam e `const` para referências que não serão reatribuídas.

let e const

```
const nomeCampo =  
document.querySelector("#nome")  
;  
  
let contador = 0;  
contador = contador + 1;
```

[editar](#) → [salvar](#) → [testar](#)

[Link: MDN Variables](#)

Tipos de dados comuns

Tópico: Texto, números e booleanos

JavaScript trabalha com valores de diferentes tipos, como strings para textos, numbers para cálculos e booleans para verdadeiro ou falso. Em interações simples, esses tipos aparecem ao ler campos, contar caracteres, verificar aceite de termos ou decidir se uma mensagem deve aparecer.

Conceitos-chave

'''

string

123

number

V/F

boolean

[]

array

tipos orientam decisões

[Link: MDN Grammar and Types](#)

Funções

Tópico: Organizar instruções reutilizáveis

Função é um bloco de código com nome e responsabilidade definida. Ela evita repetição, melhora a leitura e ajuda a dividir problemas. Em uma página, podemos criar funções para abrir menu, validar campos, calcular total ou exibir mensagens ao usuário.

Função simples

```
function mostrarMensagem(texto)
{
    const aviso =

    document.querySelector("#aviso"
);
    aviso.textContent = texto;
}

mostrarMensagem("Pronto!");
```

editar → salvar → testar

[Link: MDN Functions](#)

Parâmetros e retorno

Tópico: Entrada e saída de uma função

Uma função pode receber parâmetros e devolver resultado com `return`. Isso permite separar cálculo e apresentação. Por exemplo, uma função calcula se a senha tem tamanho suficiente, enquanto outra decide como mostrar a mensagem na tela.

`return`

```
function senhaValida(senha) {  
  return senha.length >= 8;  
}  
  
const ok =  
  senhaValida("12345678");
```

[editar](#) → [salvar](#) → [testar](#)

Estruturas condicionais

Tópico: Tomar decisões no código

Condicionais permitem executar caminhos diferentes conforme uma situação. Em páginas web, usamos `if`, `else if` e `else` para decidir mensagens, validar regras, alternar estilos ou impedir uma ação quando os dados ainda não estão adequados.

if / else

```
if (idade >= 18) {  
    mensagem = "Acesso  
permitido";  
} else {  
    mensagem = "Verifique a  
idade";  
}
```

[editar](#) → [salvar](#) → [testar](#)

[Link: MDN Conditionals](#)

Operadores de comparação e lógica

Tópico: Construir condições

Comparações como `===`, `!==`, `>` e `<` ajudam a verificar valores. Operadores lógicos como `&&` e `||` combinam condições. Em formulários, isso permite conferir se um campo foi preenchido e se outra regra também foi respeitada antes de liberar uma ação.

Condição combinada

```
const emailOk =  
  email.includes("@");  
const senhaOk = senha.length >=  
  8;  
  
if (emailOk && senhaOk) {  
  enviar = true;  
}
```

[editar](#) → [salvar](#) → [testar](#)

Estruturas de repetição

Tópico: Executar uma ação várias vezes

Laços de repetição percorrem listas de elementos, dados ou opções. Em front-end, podem ser usados para criar itens na tela, verificar campos de um formulário ou aplicar classes. O `for` é comum quando sabemos percorrer uma coleção em sequência.

for

```
const campos =  
  
document.querySelectorAll("input");  
  
for (const campo of campos) {  
  console.log(campo.name);  
}
```

editar → salvar → testar

[Link: MDN for statement](#)

DOM: a página como árvore

Tópico: Selecionar e alterar elementos

O DOM representa o documento HTML como uma árvore de objetos. Com JavaScript, selecionamos elementos usando métodos como `querySelector` e alteramos texto, classes, atributos ou estilos. Essa manipulação deve ser usada com cuidado para manter clareza e acessibilidade.

Selecionar elemento

```
const titulo =  
  document.querySelector("h1");  
  
titulo.textContent =  
  "Página atualizada";
```

editar → salvar → testar

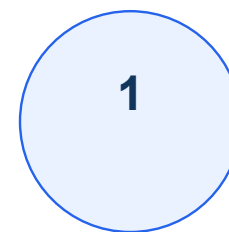
[Link: MDN DOM Introduction](#)

Eventos

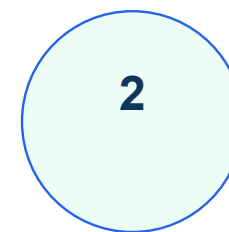
Tópico: Quando algo acontece na interface

Evento é uma ocorrência que o navegador informa ao código, como clique, digitação, envio de formulário ou mudança de seleção. Com `addEventListener`, registramos uma função que será executada quando o evento acontecer, tornando a página interativa.

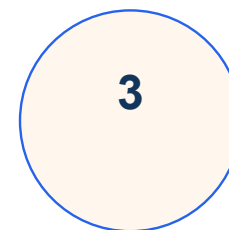
Evento no navegador



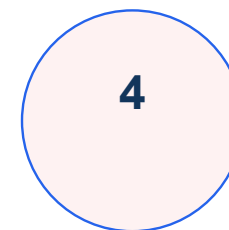
usuário clica



evento ocorre



função reage



DOM muda

interação = código respondendo ao usuário

[Link: MDN Events](#)

Clique em botão

Tópico: Primeira interação JavaScript

Uma interação simples pode exibir ou ocultar uma área de informações. O botão recebe um ouvinte de evento, e a função altera uma classe CSS no elemento alvo. Essa prática integra HTML semântico, estilo CSS e comportamento JavaScript.

click

```
const botao =  
  
document.querySelector("#detalhesBtn");  
const painel =  
  
document.querySelector("#detalhes");  
  
botao.addEventListener("click",  
() => {  
  
    painel.classList.toggle("aberto");  
});
```

[editar](#) → [salvar](#) → [testar](#)

Evento de formulário

Tópico: Interceptar envio para validar

O evento submit ocorre quando o usuário tenta enviar o formulário. Em protótipos locais, podemos impedir temporariamente o envio com `preventDefault` e mostrar uma mensagem. Em sistemas reais, a validação no cliente melhora a experiência, mas o servidor ainda deve validar.

submit

```
form.addEventListener("submit",  
(e) => {  
    e.preventDefault();  
    mensagem.textContent =  
        "Formulário conferido.";  
});
```

[editar](#) → [salvar](#) → [testar](#)

Boas práticas com eventos

Tópico: Interação previsível

Eventos devem melhorar a experiência sem esconder funcionalidades essenciais. Prefira elementos nativos, como button e form, antes de simular comportamento com div. Mantenha foco visível, mensagens textuais e estados claros para que a interação funcione por mouse, toque e teclado.

Conceitos-chave

HTML

elemento nativo

JS

evento claro

CSS

estado visível

Tab

teclado

previsibilidade melhora uso

[Link: W3C WCAG 2.2](#)

Estrutura de arquivos do projeto

Tópico: Organização mínima

Para a prática, o projeto pode usar três arquivos principais: `index.html` para estrutura, `style.css` para apresentação e `script.js` para comportamento. Essa divisão reforça a separação de responsabilidades e facilita encontrar erros durante a construção do protótipo.

Projeto web

1. `index.html`

2. `style.css`

3. `script.js`

4. abrir no navegador

5. testar e corrigir

Passo 1: criar HTML responsivo

Tópico: Base semântica

Comece criando a estrutura mínima do HTML com lang, charset, meta viewport e link para o CSS. Em seguida, organize header, nav, main, section e footer. A responsividade começa com conteúdo bem estruturado, não apenas com regras visuais.

HTML base

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
content="width=device-width,
initial-scale=1.0">
  <link rel="stylesheet"
href="style.css">
</head>
```

[editar](#) → [salvar](#) → [testar](#)

Passo 2: criar navegação clara

Tópico: Links internos e ordem lógica

Monte um cabeçalho com título e nav contendo links para seções internas. Use textos descritivos, como Serviços, Agendamento e Contato. Em telas pequenas, a navegação pode mudar de linha ou virar coluna sem perder a ordem de leitura.

Navegação

```
<header>
  <h1>Agenda Fácil</h1>
  <nav class="menu">
    <a
href="#servicos">Serviços</a>
    <a
href="#agenda">Agendamento</a>
    <a
href="#contato">Contato</a>
  </nav>
</header>
```

editar → salvar → testar

Passo 3: aplicar CSS mobile first

Tópico: Estilo inicial para telas pequenas

Escreva primeiro regras simples para celular: conteúdo em uma coluna, largura fluida, espaçamentos confortáveis e botões ocupando largura adequada. Depois acrescente melhorias para telas maiores, como menu horizontal, grid de cards e seções lado a lado.

CSS inicial

```
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
}  
.container {  
  width: min(92%, 960px);  
  margin: auto;  
}  
.menu {  
  display: flex;  
  flex-direction: column;  
}
```

[editar](#) → [salvar](#) → [testar](#)

Passo 4: melhorar telas maiores

Tópico: Media query com min-width

Depois que a versão pequena estiver funcional, use media query com min-width para ajustar telas maiores. O menu pode ficar em linha, uma seção pode virar grid e os cards podem dividir espaço. A página ganha organização sem duplicar conteúdo.

Tela maior

```
@media (min-width: 768px) {  
  .menu {  
    flex-direction: row;  
    justify-content: flex-end;  
  }  
  .cards {  
    display: grid;  
    grid-template-columns:  
    repeat(3, 1fr);  
  }  
}
```

[editar](#) → [salvar](#) → [testar](#)

Passo 5: criar primeira interação

Tópico: Mostrar e ocultar conteúdo

Crie um botão para mostrar detalhes de uma seção, como horários de atendimento ou regras do serviço. No HTML, use `button`; no CSS, controle a classe escondida; no JavaScript, alterne a classe ao clicar. Essa interação é simples e útil.

toggle

```
const btn =  
document.querySelector("#btnInfo");  
const info =  
document.querySelector("#info")  
;  
  
btn.addEventListener("click",  
  () => {  
  
    info.classList.toggle("oculto")  
    ;  
  });
```

[editar](#) → [salvar](#) → [testar](#)

Passo 6: criar segunda interação

Tópico: Mensagem dinâmica no formulário

A segunda interação pode mostrar uma mensagem quando o usuário seleciona um serviço ou preenche um campo. Isso demonstra leitura de valor, uso de variável, condicional e atualização do DOM, conectando vários fundamentos de programação front-end em um exemplo pequeno.

change + if

```
servico.addEventListener("change", () => {  
  const escolhido =  
    servico.value;  
  if (escolhido === "") {  
    aviso.textContent =  
      "Escolha um serviço.";  
  } else {  
    aviso.textContent =  
      `Serviço: ${escolhido}`;  
  }  
});
```

[editar](#) → [salvar](#) → [testar](#)

Passo 7: testar adaptação

Tópico: Checklist prático

Teste a página em largura pequena, média e grande. Verifique se não há rolagem horizontal, se a navegação permanece acessível, se textos não ficam espremidos, se botões continuam visíveis e se as interações funcionam por clique, toque e teclado.

Auditoria rápida

1. sem rolagem horizontal

2. texto legível

3. menu alcançável

4. foco visível

5. interações funcionando

Exemplo aplicado: Agenda Fácil

Tópico: Estudo de caso integrador

No caso Agenda Fácil, a página pode apresentar serviços de um salão, horários de funcionamento, formulário de solicitação e navegação entre seções. O layout responsivo organiza cards de serviços no desktop e empilha informações no celular, mantendo o formulário acessível.



Exemplo aplicado: Biblioteca Bairro Aberto

Tópico: Catálogo e solicitação

No caso da biblioteca, cards podem representar livros disponíveis, filtros podem destacar categorias e um formulário pode solicitar empréstimo. Em tela grande, o catálogo pode usar grade; em celular, os cards devem aparecer em sequência, com botões grandes e textos legíveis.



Erros comuns em responsividade

Tópico: O que evitar

Erros frequentes incluem definir largura fixa para tudo, esconder conteúdo importante no celular, remover foco do teclado, usar imagem sem limite de largura, criar menu que só funciona por mouse e escolher breakpoints sem observar o comportamento real do conteúdo.

Conceitos-chave

px

largura fixa

img

imagem estoura

Tab

sem foco

≡

menu frágil

corrigir testando no navegador

Erros comuns em JavaScript inicial

Tópico: Diagnóstico rápido

Em projetos iniciais, aparecem erros como selecionar id inexistente, executar script antes do HTML carregar, esquecer ponto e vírgula em lugares importantes, confundir `=` com `===`, duplicar código e criar funções sem nome claro ou sem responsabilidade definida.

Erros típicos

```
// id não existe no HTML
const btn =
document.querySelector("#botao"
);

// cuidado: atribuição não é
comparação
if (valor = "ok") {
    // erro lógico
}
```

[editar](#) → [salvar](#) → [testar](#)

Acessibilidade em layouts responsivos

Tópico: Não quebrar a navegação

Uma página responsiva precisa continuar acessível em cada breakpoint. Isso significa manter ordem de foco coerente, texto visível, contraste adequado, controles clicáveis ou tocáveis, mensagens compreensíveis e navegação por teclado. A adaptação visual não pode criar barreiras novas.

Teste por breakpoint

1. desktop

2. tablet

3. celular

4. teclado

5. leitor/estrutura

[Link: W3C WCAG 2.2](#)

Miniatividade em sala

Tópico: Planejar layout e interação

Em dupla, escolha um estudo de caso e defina três seções da página, uma navegação, uma mudança responsiva e duas interações JavaScript. A entrega oral deve justificar por que cada decisão ajuda o usuário em computador e em celular.

Planejamento

1. escolher caso

2. definir seções

3. planejar menu

4. planejar breakpoints

5. definir interações

Gabarito comentado da miniatividade

Tópico: Uma solução possível

Uma solução para Agenda Fácil pode usar header com menu, seções de serviços, horários e contato, cards em grid no desktop e lista no celular. As interações podem abrir detalhes de serviço e mostrar mensagem dinâmica ao escolher um horário no formulário.



Roteiro para atividade prática

Tópico: Passo a passo consolidado

O aluno deve criar a pasta do projeto, escrever HTML semântico, conectar CSS e JavaScript, implementar layout mobile first, acrescentar media query, criar duas interações, testar no DevTools, revisar acessibilidade básica e registrar evidências para o relatório.

Roteiro final

1. criar arquivos

2. HTML semântico

3. CSS responsivo

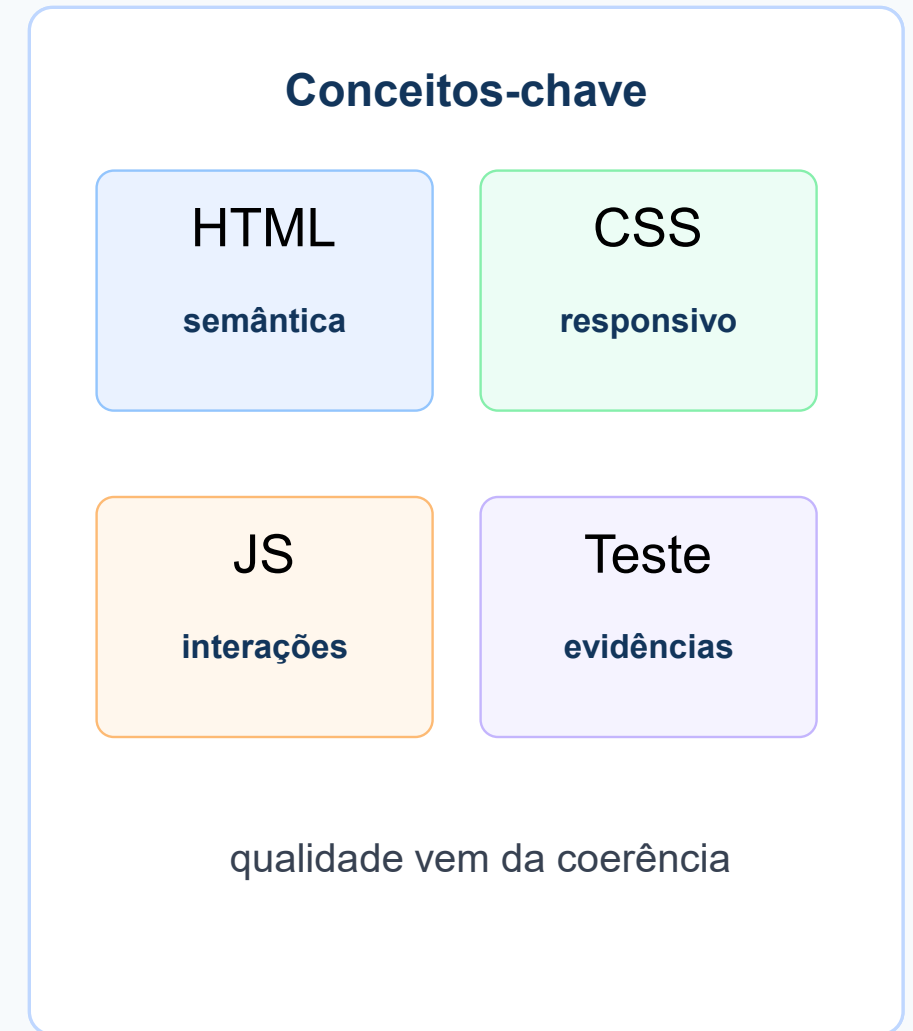
4. JS com eventos

5. testar e documentar

Critérios de qualidade do protótipo

Tópico: Como avaliar a entrega

Um bom protótipo não precisa ser complexo, mas deve ser coerente. A página precisa ter estrutura semântica, navegação clara, layout adaptável, CSS organizado, interações JavaScript funcionais, formulário compreensível e evidências de teste em telas diferentes.



Síntese da aula

Tópico: Ideias principais

Layouts responsivos unem conteúdo bem estruturado, CSS flexível, media queries e testes reais em diferentes larguras. JavaScript acrescenta comportamento por meio de variáveis, funções, condicionais, repetições e eventos. O resultado esperado é uma página útil, acessível e fácil de manter.

Do código ao uso

1. estrutura

2. apresentação

3. adaptação

4. interação

5. teste

Referências e materiais recomendados

Tópico: Fontes para estudo complementar

Material anexo: slides de formulários, validação, acessibilidade e CSS; exercício prático resolvido de formulário; slides de estrutura HTML; orientações da avaliação processual. MDN: Responsive Design, Media Queries, Flexbox, Grid, JavaScript, Variables, Functions, Conditionals e Events. W3C: WCAG 2.2 e Quick Reference. web.dev: Responsive Web Design Basics. Sugestões de vídeo: Curso em Vídeo JavaScript, Kevin Powell CSS responsivo e Google Chrome Developers DevTools.

Conceitos-chave

MDN

documentação

W3C

acessibilidade

web.dev

responsivo

YT

vídeos

consultar fontes oficiais